Übungsblatt 7

RESTful API-Design, HATEOAS & SSL/TLS

5430UE Web and Data Engineering

27. Mai 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs



Lernziele

- Grundlegendes Verständnis der HTTP-Semantik zur Nutzung von Methoden und Status-Codes.
- Einblick in ressourcenorientiertes URL-Design und die Konzepte von HATEOAS.
- Kenntnis der wesentlichen Unterschiede zwischen REST- und SOAP-Architekturen.
- Basiswissen zur Transport-Sicherheit mittels TLS und dem Ablauf des Handshakes.

HTTP-Methoden und Status-Codes

HTTP-Methoden und Status-Codes(1/2)

In dieser Aufgabe betrachten wir die HTTP-Methoden POST, GET, PUT, PATCH, DELETE, HEAD, OPTIONS.

1. Beschreiben Sie den Zweck jeder der Methoden in einem Satz. Geben Sie jeweils an, ob die Methode sicher und/oder idempotent ist.

HTTP-Methoden und Status-Codes(2/2)

In dieser Aufgabe betrachten wir die HTTP-Methoden POST, GET, PUT, PATCH, DELETE, HEAD, OPTIONS.
2. Ordnen Sie jeder Situation die korrekte HTTP-Methode und passenden Status-Code zu:

Situation	Methode	Status-Code
Produktliste abrufen	?	?
Neues Produkt anlegen	?	?
Produktname aktualisieren (nur ein Feld)	?	?
Produkt vollständig ersetzen	?	?
Produkt löschen	?	?
Ressource nicht gefunden	-	?
Validierungsfehler im Request-Body	-	?
Server-interner Fehler	-	?

Caching-Strategien

Caching-Strategien

Ein Online-Shop liefert verschiedene Ressourcen. Bestimmen Sie für jede die geeignete Caching-Strategie:

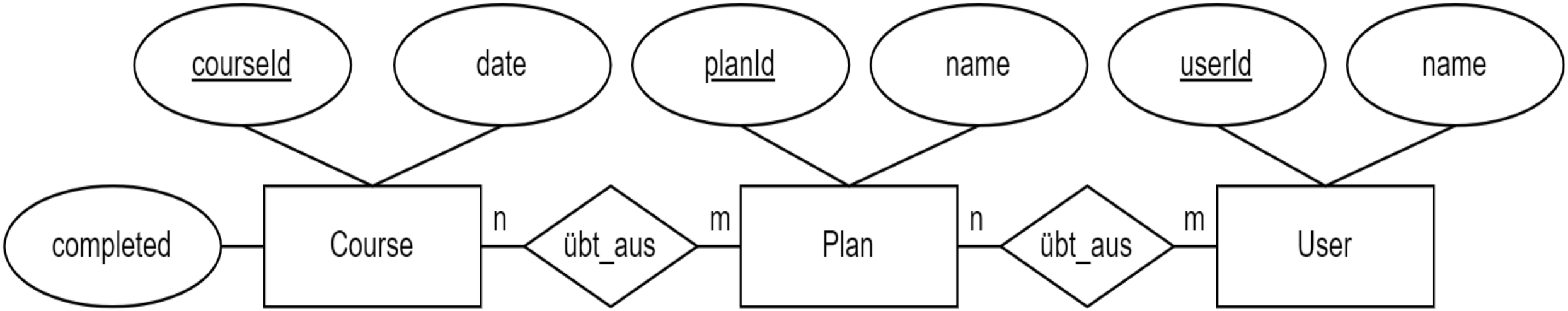
Ressource	Strategie (kein Cache / kurz / lang / ewig)	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	?	?
Produktliste (aktualisiert sich stündlich)	?	?
Warenkorb-Inhalt (nutzerspezifisch)	?	?
JavaScript-Bundle `app.abc123.js` (Hash im Namen)	?	?

RESTful URL-Design



RESTful URL-Design (1/3)

Gefordert ist der Entwurf eines RESTful WebService zur Verwaltung von Trainingsplänen eines Fitnesscenters. Nehmen Sie dazu folgendes (vereinfachtes) Datenschema an:



Ein Trainingsplan beinhaltet mehrere Übungen, und mehrere Personen können denselben Trainingsplan ausüben.

RESTful URL-Design (2/3)

Ein Backend soll folgende Operationen unterstützen:

1. Alle Trainingspläne eines Nutzers abrufen
2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen
3. Alle Personen anzeigen, die einen gewissen Namen haben
4. Alle abgeschlossen Kurse eines Nutzers an einen spezifischen Tag filtern
5. Einen neuen Trainingsplan für einen Nutzer erstellen
6. Einen Kurs in einem Trainingsplan aktualisieren
7. Einen Kurs mit gegebener Id löschen



RESTful URL-Design (3/3)

1. Entwerfen Sie ein URL Mapping für einen RESTful WebService für die oben genannten 7 Funktionen. Bestimmen Sie jeweils die HTTP Methode und URL, die ein Client aufzurufen hat, um die Funktion auszuführen. Halten Sie sich für das URL Design sowie die Wahl der HTTP Methoden an die Konventionen aus der Vorlesung. Kennzeichnen Sie dynamische Pfadelemente mit geschweiften Klammern ({}). Die Angabe von Protokoll und Domain können Sie mit "fitness.de" abkürzen. Sollten für eine Anfrage etwaige Parameter mit übertragen werden müssen, geben Sie nur Parameter in der URL an (der Inhalt des Request Body muss nicht mit angegeben werden).
2. Welche der URL-Muster sind dabei problematisch:
   - `/getTrainings`
   - `/user/deleteById?id=5`



HATEOAS und API-Versionierung

HATEOAS und API-Versionierung

1. **HATEOAS**:
    - Was bedeutet HATEOAS?
    - Welchen praktischen Vorteil bietet es gegenüber einer reinen Daten-API?
    - Zeigen Sie ein JSON-Beispiel für eine Produktressource mit HATEOAS-Links.
2. Nennen Sie zwei Strategien für **API-Versionierung** und diskutieren Sie je einen Vorteil und Nachteil.

SOAP vs REST

SOAP vs REST

1. Erklären Sie REST und SOAP kurz in jeweils 1-2 Sätzen.
2. Entscheiden Sie für jedes Szenario, ob REST oder SOAP besser geeignet ist, und begründen Sie:

Szenario	REST oder SOAP	Begründung
Mobile App ruft Produktdaten ab	?	?
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)	?	?
IoT-Sensoren senden Messwerte (geringe Bandbreite)	?	?
Legacy-Enterprise-Integration mit strenger Vertragsbindung	?	?

TLS/SSL - Transport-Sicherheit

TLS/SSL - Transport-Sicherheit

1. Was war das zentrale Sicherheitsproblem von SSL 2.0, das zur Entwicklung von TLS führte?
2. Ist TLS heute für Web-Anwendungen verpflichtend oder optional? Begründen Sie aus technischer und regulatorischer Sicht.
3. Erklären Sie den TLS-Handshake in 4 Schritten: Was wird ausgetauscht und welches Ziel hat jeder Schritt?
4. Was bedeutet Certificate Pinning und für welche Art von Anwendung ist es besonders relevant?